

Tiny bit of fork-join, Priority queues

5/14/25

Administrivia

- HW 8 (recursive list methods) due tomorrow night

Recall: Fork-join in Java

```
class Sum extends RecursiveTask<Integer> {  
    //attributes and constructor  
  
    protected Integer compute(){  
        if(hi - low < THRESHOLD) return simpleSum(array, low, hi);  
        Sum left = new Sum(array, low, (hi+low)/2);  
        Sum right= new Sum(array, (hi+low)/2, hi);  
        left.fork();  
        return right.compute() + left.join();  
    }  
}  
  
//start with this (waits for result):  
sum = ForkJoinPool.commonPool().invoke(new Sum(array, 0, array.length));
```

How do we sum the values in an $n \times n$ array?

Priority Queue ADT

- Store collection of values and read them out in priority order (as given by a Comparator)
- Operations:
 - void enqueue(value) //add value to collection
 - value dequeue() //remove & return next val
 - value peek() //return next val w/o removing
 - int size() //get collection size

Application: HPC system simulation

- Simulating large computer system
 - 10,000+ processors in a room w/ fast network
 - Programs (“jobs”) run on many processors at once
 - Jobs submitted to a “queue” w/ a number of processors and an estimated running time
- Priority queue of events (job submission and completion)

Application: Encoding data

- How many bits does it take to encode 100 characters?

Application: Encoding data

- How many bits does it take to encode 100 characters?
 - What if you know they are only a thru e?

Application: Encoding data

- How many bits does it take to encode 100 characters?
 - What if you know they are only a thru e?
 - What if you know they have a specific frequency?

char	a	b	c	d	e
#times	47	11	20	5	17

Huffman encoding

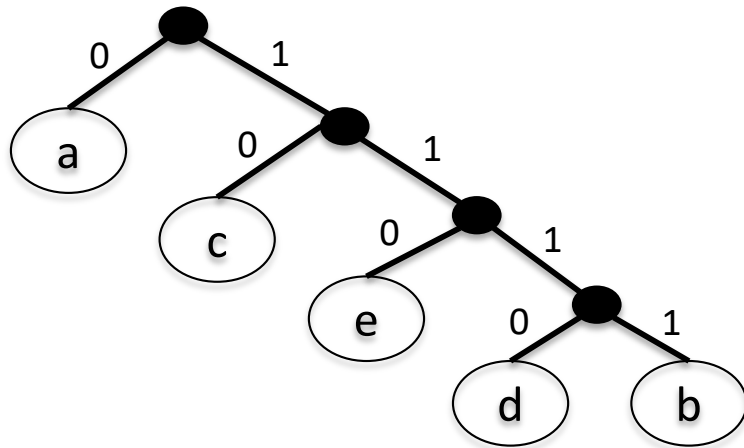
char	a	b	c	d	e
#times	47	11	20	5	17
encoding	0	1111	10	1110	110

- Length: $47 + 2 * 20 + 3 * 17 + 4 * (11 + 5) = 202$

Huffman encoding

char	a	b	c	d	e
#times	47	11	20	5	17
encoding	0	1111	10	1110	110

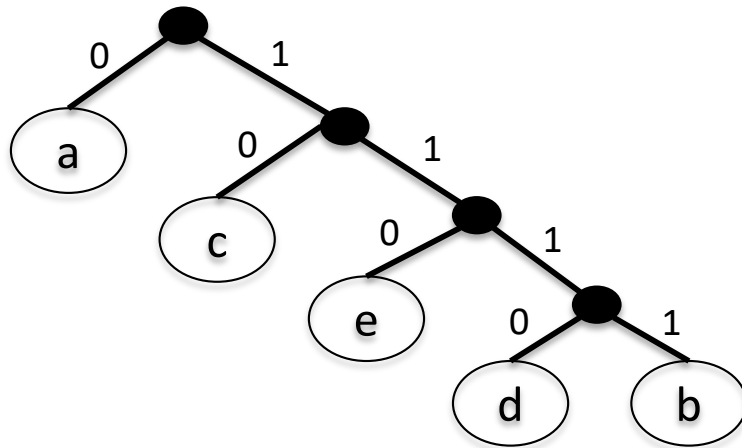
- Length: $47 + 2 * 20 + 3 * 17 + 4 * (11 + 5) = 202$



Huffman encoding

char	a	b	c	d	e
#times	47	11	20	5	17
encoding	0	1111	10	1110	110

- Length: $47 + 2*20 + 3*17 + 4*(11+5) = 202$



Start with a node for each char
While there is more than one node,
Combine 2 nodes w/ lowest frequencies
//Combined node has sum of frequencies

Suppose a Priority Queue is implemented using a **unsorted** linked list. Which of the following are the running times of enqueue and dequeue respectively?

- A. $O(1)$ and $O(\log n)$
- B. $O(1)$ and $O(n)$
- C. $O(n)$ and $O(1)$
- D. $O(n)$ and $O(n)$
- E. None of the above

Suppose a Priority Queue is implemented using a **unsorted** linked list. Which of the following are the running times of enqueue and dequeue respectively?

- A. $O(1)$ and $O(\log n)$
- B. $O(1)$ and $O(n)$
- C. $O(n)$ and $O(1)$
- D. $O(n)$ and $O(n)$
- E. None of the above

Suppose a Priority Queue is implemented using a **sorted** linked list (highest priority first). Which of the following are the running times of enqueue and dequeue respectively?

- A. $O(1)$ and $O(\log n)$
- B. $O(1)$ and $O(n)$
- C. $O(n)$ and $O(1)$
- D. $O(n)$ and $O(n)$
- E. None of the above

Suppose a Priority Queue is implemented using a **sorted** linked list (highest priority first). Which of the following are the running times of enqueue and dequeue respectively?

- A. $O(1)$ and $O(\log n)$
- B. $O(1)$ and $O(n)$
- C. $O(n)$ and $O(1)$
- D. $O(n)$ and $O(n)$
- E. None of the above

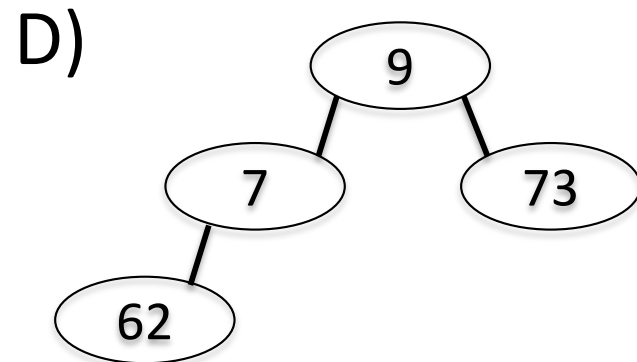
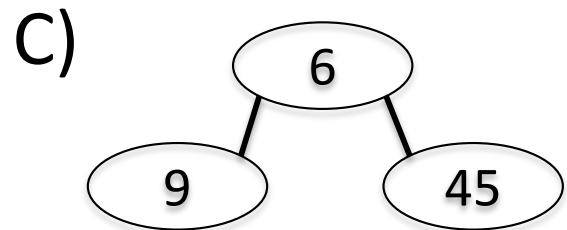
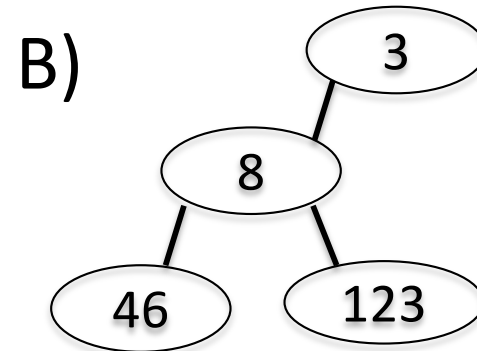
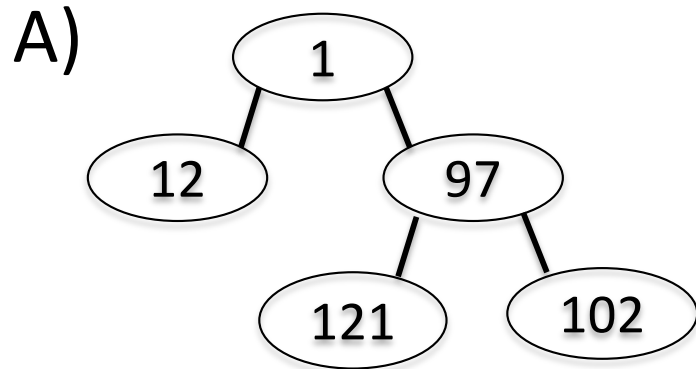
First implementation ideas

- Linked list sorted by priority:
 - $O(n)$ enqueue, $O(1)$ dequeue
- Unsorted linked list:
 - $O(1)$ enqueue, $O(n)$ dequeue

Heaps

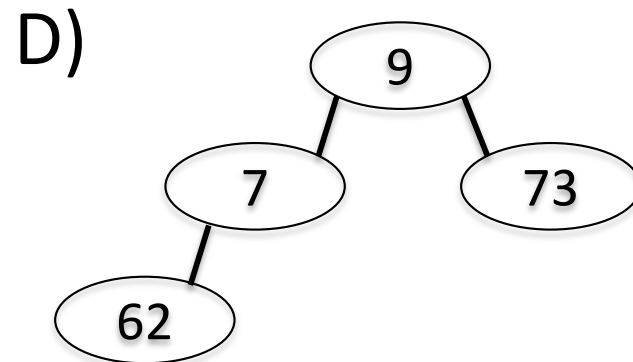
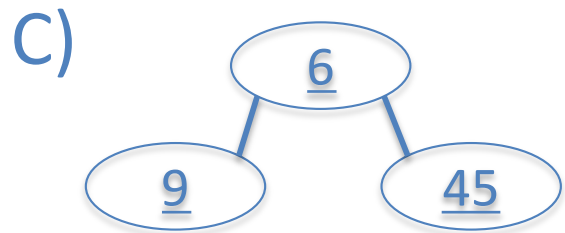
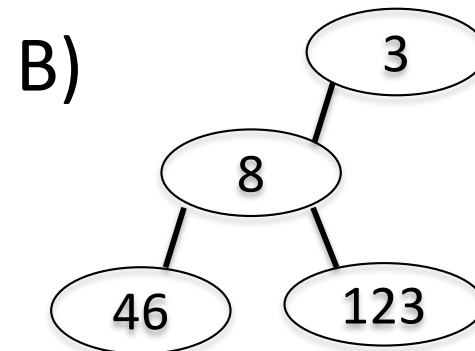
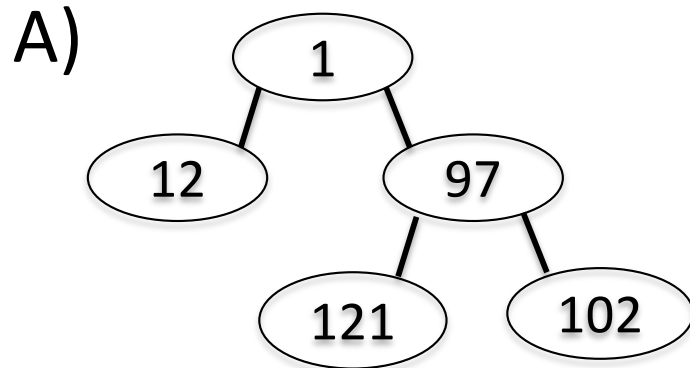
- A heap is a data structure specifically for priority queues
- Displayed as a complete binary tree (two children for every node except possibly nodes on the right of the bottom level)
- Each parent is $\leq$ either child (by Comparator)

Which of the following is a valid heap?



E) Not exactly one of the above

Which of the following is a valid heap?



E) Not exactly one of the above

Efficient representation of a heap

- Values stored in an array indexed from 1
- Root stored in index 1
- Can move thru the tree via index arithmetic:
 - left child of value in index i : index $2i$
 - right child of value in index i : index $2i+1$
 - parent of value in index i : index $i/2$ (round down)

Which of the following is a valid heap?

A.

1	2	3	4	5	6	7	8	9
4	5	6	7	8	12	18	142	215

B.

1	2	3	4	5	6	7	8	9
4	6	5	12	8	142	7	18	215

C.

1	2	3	4	5	6	7	8	9
4	7	5	142	12	8	6	18	215

D.

1	2	3	4	5	6	7	8	9
4	8	5	12	142	6	215	7	18

E. Not exactly one of the above

Which of the following is a valid heap?

A.

1	2	3	4	5	6	7	8	9
4	5	6	7	8	12	18	142	215

B.

1	2	3	4	5	6	7	8	9
4	6	5	12	8	142	7	18	215

C.

1	2	3	4	5	6	7	8	9
4	7	5	142	12	8	6	18	215

D.

1	2	3	4	5	6	7	8	9
4	8	5	12	142	6	215	7	18

E. Not exactly one of the above (A and B)